

Haedal Protocol Audit Report

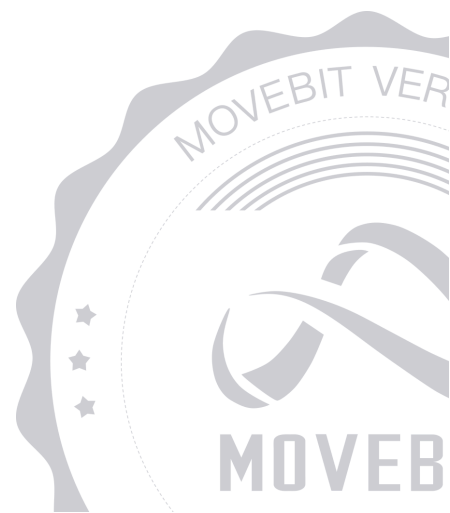


contact@bitslab.xyz



https://twitter.com/movebit_

Wed Sep 23 2026



Haedal Protocol Audit Report

1 Executive Summary

1.1 Project Information

Description	A Sui Move DeFi vault that deposits user assets into multiple yield protocols
Type	Staking
Auditors	Alex,Sprig,Bear Two
Timeline	Tue Jun 23 2026 - Wed Sep 23 2026
Languages	Move
Platform	Sui
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/haedallsd/lending-vaults
Commits	9c8de1180877e6a32f7240a7b51f2f7cd024fb65 c4a5b7e4123256eeac424de274f061f1c0175a87 212dac151f8afdd9602261139de5cbcce0730df8

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
MOV1	contracts/htoken/Move.toml	3e4c14b9df23570788cdce4c260366261e36a2b5
MOV6	contracts/farming/Move.toml	472a571c906f8c706fa0cb6cdbb4d5c178c1bab0
FMA	contracts/farming/sources/farming_math.move	43e015d4edff046e1d8d93443f4856716c04cab2
VAT	contracts/llv_ext_vault/sources/vault_adapter_tests.move	0e35986f1fbfec5c90a7a1b1f81c92587153ba9
VAD	contracts/llv_ext_vault/sources/vault_adapter.move	fc022734ad06c0cf59c7c57777cc729dcc44dbe9
HUS	new/htoken/sources/husdc.move	50f07d46d29f5ee0b8311052baa5b40aeeaa555c
FEV	new/farming/sources/farming_events.move	cfb63a614f50da77808743e9e4157747be4f50c0
PUB	contracts_v2/adapter_cetus_sui/Published.toml	9aa4af5515035c93269d49a78e060caef55c6b1f
MOV	contracts_v2/adapter_cetus_sui/Move.toml	b2dd10caf4b2b0099908c864d47e420361201c59
CHE	contracts_v2/adapter_cetus_sui/sources/cetus_hasui_entry.move	e324f50f98c7c1ef08289d762f5f2e2e6053f773

CSO	contracts_v2/adapter_cetus_sui/sources/cetus_sui_oracle.move	dfaec930f2112d76653b845a62a15b1f132ca8e2
CVA	contracts_v2/adapter_cetus_sui/sources/cetus_vault_adapter.move	7248ca23610f45b8121cf4d3ef64da5255cdb516
CCO	contracts_v2/adapter_cetus_sui/sources/cetus_common.move	1545f4ca5f76615a98785ee757bbbc5ee77102f4
CVAT	contracts_v2/adapter_cetus_sui/sources/cetus_vault_adapter_tests.move	6dc2ae02b6fae9339e0ce7f30baace8bb12f08eb
CAE	contracts_v2/adapter_cetus_sui/sources/cetus_admin_entry.move	139e5ccbeb480e1731851700213c6e9487bf7ed3
CPO	contracts_v2/adapter_cetus_sui/sources/cetus_position.move	cb75138d300a6b14e2a821bfdff96dd454f7aa27
CEN	contracts_v2/adapter_cetus_sui/sources/cetus_entry.move	956409f211040c333a9852899eff92e2692e2eef
PUB1	contracts_v2/farming/Published.toml	a12c8b9c5d685e6d53f57801835fb3c17c4f524d
FAR	contracts_v2/farming/sources/farming.move	14e4a687b7890de1acb5631023bbf38dc7493c68
PUB2	contracts_v2/haedal_vault/Published.toml	f187e094c6d5509b6252c12e4c2d34d8684f2034
MOV2	contracts_v2/haedal_vault/Move.toml	2f36ec60693d8519ca4c0bfa0e29addc4d821b80
VEM	contracts_v2/haedal_vault/sources/vault_emergency.move	098ae5f4a35fada302348f6d5d049a331385fc5a

VDE	contracts_v2/haedal_vault/sources/ vault_deposit.move	8da19a044a170c97aa3413ebfb8b8 6a5cfba3638
VIN	contracts_v2/haedal_vault/sources/ vault_incentive.move	990e6b7e03c82345d7a0797a184a 7314d7da587a
VAC	contracts_v2/haedal_vault/sources/ vault_acl.move	6763558b762b8b6db25db005f3d3 88c1e45e39db
VPO	contracts_v2/haedal_vault/sources/ vault_pool.move	a6d98685fc68e293a3b40e76281d2 7fe6b945c10
VSY	contracts_v2/haedal_vault/sources/ vault_sync.move	8ed37548594ab63431cde7ec2020f c9a8bd9e281
VRE	contracts_v2/haedal_vault/sources/ vault_rebalance.move	ac585c779d994e117a8e0f07e31c1 e42526ab761
VVA	contracts_v2/haedal_vault/sources/ vault_validation.move	bb93dd08c6e93fa4a7f20ac1d28c4 74c886003e3
VMA	contracts_v2/haedal_vault/sources/ vault_math.move	83b428418742af12ef7aa3132aef10 6c2829e935
VEN	contracts_v2/haedal_vault/sources/ vault_entry.move	07a936d454dedbad499691546bb8 bffc10812570
VHO	contracts_v2/haedal_vault/sources/ vault_holdings.move	8dbbd174e2de2badfdc9632ae845 cd57a240a128
VAD	contracts_v2/haedal_vault/sources/ vault_admin.move	627f9f4edc7c46d53a945bd7ad9e4 4a97972cee7
VST	contracts_v2/haedal_vault/sources/ vault_strategy.move	c4cbd72a871da3746b48e96dcdf6c 3fbf7028a66

VCCP	contracts_v2/haedal_vault/sources/ vault_cetus_clmm_params.move	36c7caafeb5548a1b231980d6198e 6c0bb80986d
VWI	contracts_v2/haedal_vault/sources/ vault_withdraw.move	101b053fee072f0b14dfc0d5fd111b bc6a60a018
VAE	contracts_v2/haedal_vault/sources/ vault_admin_entry.move	3aec3e10b14d1b356a91bccbea2 e1fff5c061c8
VEV	contracts_v2/haedal_vault/sources/ vault_events.move	79c20ae059c54d0d714c4f6279cd3 86b6450f098
PUB3	contracts_v2/adapter_navi/Publish ed.toml	145b085eb15b9a27a22ee3122680 d8256860821e
MOV3	contracts_v2/adapter_navi/Move.to ml	fd46b6650b5579030d63deb24d27 400606fc632b
NAT	contracts_v2/adapter_navi/source s/navi_adapter_tests.move	a6d0c50ab227639bd53e9c362b89 6cccad1c373f
NAD	contracts_v2/adapter_navi/source s/navi_adapter.move	1c980a5759561bcf3dc64170428af a41df4f3567
NEN	contracts_v2/adapter_navi/source s/navi_entry.move	dacee0ab033de6a6719ed1956036 4e4971c45016
NAE	contracts_v2/adapter_navi/source s/navi_admin_entry.move	8f77ddd7896ebb355e37a48864b8 a7ddb63cb02c
MOV4	contracts_v2/llv_ext_vault/Move.to ml	3f5b79597e8ea106574f719613250 9f16c434019
VEN1	contracts_v2/llv_ext_vault/sources/ vault_entry.move	50389901fe6046647008b774ffb2d 5b9ce51d33f

VAE1	contracts_v2/llv_ext_vault/sources/ vault_admin_entry.move	799e2fddf2e86135255ef64204f729 2534635b5f
PUB4	contracts_v2/adapter_scallop/Publi shed.toml	89952d17a1e077ae3b4059c8b0efc 9d1df34f282
MOV5	contracts_v2/adapter_scallop/Mov e.toml	d62b3424024c3defc4eea479f2dd5 741dd66507b
SSE	contracts_v2/adapter_scallop/sour ces/scallop_scoin_entry.move	af972f05a275cf8a0f4d92cdf7f2a54 c36feb26d
SAD	contracts_v2/adapter_scallop/sour ces/scallop_adapter.move	e259eacfe03c0898caa7d61ab750c e540a7275da
SCO	contracts_v2/adapter_scallop/sour ces/scallop_common.move	2e0c90296e1656c43a53e391342c9 006b3f94150
SAT	contracts_v2/adapter_scallop/sour ces/scallop_adapter_tests.move	a0bc4d576642e6ffc23d9a4df2e739 283fb03dfd
SEN	contracts_v2/adapter_scallop/sour ces/scallop_entry.move	3480f12fdb4bdf902c03ad324f5e37 c15a3b36f1
SAE	contracts_v2/adapter_scallop/sour ces/scallop_admin_entry.move	0f97cd2d8e8eb9e718308e740dae0 ed932e1fcea
PUB5	contracts_v2/adapter_cetus_wal/P ublished.toml	9b593f4d89e27a5c5476dd1dee26 3ce2d908a833
MOV7	contracts_v2/adapter_cetus_wal/M ove.toml	c040094b2f334781d8ba015476531 e18c2a10e48
CCO1	contracts_v2/adapter_cetus_wal/so urces/cetus_clmm_oracle.move	1d19dc68587cbdb6ea6739a8fb54 0cd67849516f

CCE	contracts_v2/adapter_cetus_wal/sources/cetus_clmm_entry.move	f3727bedd8a33298c8afa4e66f33eae3ffdb0c
CCP	contracts_v2/adapter_cetus_wal/sources/cetus_clmm_pool.move	d81109d8026a58710c2ad70588403a945d3f7e16
CCP1	contracts_v2/adapter_cetus_wal/sources/cetus_clmm_position.move	8e4b640fa9342f7c208b2f77a7567e5e9cd6d885
CCAE	contracts_v2/adapter_cetus_wal/sources/cetus_clmm_admin_entry.move	a6e6164e4a432dccfbd4059d41a17d1e680e8b11
CCV	contracts_v2/adapter_cetus_wal/sources/cetus_clmm_vault.move	8a458454001c8ab01c58f1a3d4116eca4f8b651b
PUB6	contracts_v2/adapter_alphalend/Published.toml	c47d4ca8b0698526532b464676140d5ff54595d7
MOV8	contracts_v2/adapter_alphalend/Move.toml	cf1be93ee965d5effacd28c12c02da084a6c5cdc
AET	contracts_v2/adapter_alphalend/sources/alphalend_entry_tests.move	13df87ee157c5972ddfd17b770e42dd9bcac9a4d
AAE	contracts_v2/adapter_alphalend/sources/alphalend_admin_entry.move	4847d89e9ce928727af1e44fc2444837ba62258f
AEN	contracts_v2/adapter_alphalend/sources/alphalend_entry.move	e1620313995b12190c0ee2e0d6ef1219e654b8df
AAT	contracts_v2/adapter_alphalend/sources/alphalend_adapter_tests.move	513c7595f67035134b16c525b76e75c88e401e6e

AAD	contracts_v2/adapter_alphalend/sources/alphalend_adapter.move	09889ea8573a787efdbbb4cf2bac8fbb2e1931d7
PUB7	contracts_v2/adapter_suilend/Published.toml	38eaf0c9012137b17615879235f74ee6be3a9e4
MOV9	contracts_v2/adapter_suilend/Move.toml	86db2838111f3cf4666d180d7436de51403012e3
SAT1	contracts_v2/adapter_suilend/sources/suilend_adapter_tests.move	e36befcb84a9e78fc9be979b4f484b7c56f70ca4
SEN1	contracts_v2/adapter_suilend/sources/suilend_entry.move	60802c757b65d4ee7d36a44c3bb337c3e1a05da7
SAE1	contracts_v2/adapter_suilend/sources/suilend_admin_entry.move	d2fa411998a257760d7a4cedbd9956000adc9768
SAD1	contracts_v2/adapter_suilend/sources/suilend_adapter.move	b36ce31a4e440bb6b0dc36d4fcd5997661570933
PUB8	contracts_v2/adapter_current/Published.toml	be1459059f795e6d2245e414173c6fd2aea04c8c
MOV10	contracts_v2/adapter_current/Move.toml	6d9b8ed0e9016a4175b685f4ec06992e560a8336
CAE1	contracts_v2/adapter_current/sources/current_admin_entry.move	1e00cc1eea0e2b6ced77374119be4af3b82a5dfd
CEN1	contracts_v2/adapter_current/sources/current_entry.move	1ecec83edcd2b2061e548937bf4899d0e096dc7
CAD	contracts_v2/adapter_current/sources/current_adapter.move	9531e70a60aa0a82247af67dcf7fce318a62c366

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	6	4	2
Critical	0	0	0
Major	1	1	0
Medium	2	2	0
Minor	3	1	2
Informational	0	0	0

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the "**Testing and Automated Analysis**", "**Code Review**" and "**Formal Verification**" strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification(Optional)

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by [Haedal](#) to identify any potential issues and vulnerabilities in the source code of the [Haedal Protocol](#) smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 6 issues of varying severity, listed below.

ID	Title	Severity	Status
FAR-1	Missing Pause Check in <code>`unstake()</code>	Minor	Acknowledged
CCO-4	Unused Functions	Minor	Fixed
FAR-6	Attackers Can Suppress Reward Distribution by Triggering Frequent Updates	Medium	Fixed
CHE-8	Manipulable On-Chain Price Weakens Withdrawal Slippage Protection	Medium	Fixed
LRE-10	Rebalance Withdraw Surplus Can Inflate Cached NAV	Major	Fixed
AEN-11	Reward Compound Receipts May Allow Reward Diversion	Minor	Acknowledged

3 Participant Process

Here are the relevant actors with their respective abilities within the [Haedal Protocol](#) Smart Contract :

Global Admin

- Global Admin can create and share a new LLV pool through the `create_and_share_pool()` function.
- Global Admin can pause or unpause the whole protocol through the `set_global_paused()` function.

Pool Admin

- Pool Admin can configure protocol object IDs through the `configure_protocol()` function.
- Pool Admin can add protocol routes through the `add_protocol()` function.
- Pool Admin can set protocol targets and queues through the `set_targets_and_queues()` function.
- Pool Admin can configure deposit limits through `set_min_deposit()` , `set_deposit_cap()` , and `set_protocol_deposit_cap()` .
- Pool Admin can configure fees through `set_fee_recipient()` , `set_performance_fee()` , and `set_management_fee()` .
- Pool Admin can manage keepers through `add_keeper()` and `remove_keeper()` .
- Pool Admin can pause or unpause a pool through `pause()` and `unpause()` .
- Pool Admin can force-refresh protocol balances through `admin_force_refresh_*`() functions.
- Pool Admin can recall protocol funds to idle through `admin_recall_*_to_idle()` functions.
- Pool Admin can inject external rewards into idle through the `inject_reward_to_idle()` function.

Keeper

- Keeper can refresh protocol balances through `refresh_*`() functions.
- Keeper can rebalance pool funds through `begin_rebalance()` , protocol `rebalance_*`() functions, and `complete_rebalance()` .
- Keeper can claim protocol rewards through `claim_*_rewards()` functions.
- Keeper can compound protocol rewards through `begin_*_rewards_for_compound()` and `complete_*_rewards_for_compound()` functions.

User

- User can deposit assets into the LLV pool through `begin_deposit()` , protocol `deposit_to_*`() functions, and `complete_deposit()` .
- User can withdraw assets from the LLV pool through `begin_withdraw()` , protocol `withdraw_from_*`() functions, and `complete_withdraw()` .
- User can perform idle-only withdrawals through the `begin_withdraw_idle_only()` function.
- User can stake LLV share tokens into farming through the `stake_after_deposit()` function.
- User can unstake farming share tokens through the `unstake_for_withdraw()` function.
- User can claim farming rewards through the `claim_farming_reward()` function.

Farming Roles

- Farming Owner can manage farming roles through `add_operator()` , `del_operator()` , `add_minior()` , `del_minior()` , `add_banker()` , and `del_banker()` .
- Farming Minor can create farming pools and configure reward schedules through `create_lend_pool()` , `create_borrow_pool()` , `add_reward_config()` , and `set_reward_config()` .
- Farming Banker can fund or extract reward-bank tokens through `fund_reward_bank()` and `extract_reward_bank()` .
- Farming Operator can pause farming pools and set boosts through `pause()` , `resume()` , and `set_boost()` .

4 Findings

FAR-1 Missing Pause Check in `unstake()`

Severity: Minor

Status: Acknowledged

Code Location:

contracts/farming/sources/farming.move#1063-1085

Descriptions:

The `unstake()` function in `farming.move` is used to unstake tokens from the farming pool. This function allows a user to withdraw their staked tokens. However, the new version of the code removed the pause check that existed in the original version. The `pause()` function clearly states that user flows like unstake should be blocked when the pool is paused. Because there is no check for `pool.is_paused` in `unstake()`, a user can still unstake tokens even when the pool is paused. This breaks the intended security control and can lead to unexpected behavior or asset loss during a paused state.

Suggestion:

It is recommended to add a check for `pool.is_paused` at the beginning of the `unstake()` function to prevent unstaking when the pool is paused.

Resolution:

The team acknowledged this issue and state that this is by design, as the pause semantics are specifically intended to block new risky operations while still allowing unstake and claim functions; the current `pause()` comments accurately reflect this behavior, so no modifications will be made to the current version.

CCO-4 Unused Functions

Severity: Minor

Status: Fixed

Code Location:

contracts/llv_ext_cetus/sources/cetus_common.move#37-45,62-70

Descriptions:

The `configured_withdraw_slippage_bps()` function and the `configured_instant_unstake_fee_rate()` function are defined in the `cetus_common` module. These two functions are marked as `public(package)`, which means they can only be called by other modules within the same package. However, after checking the entire codebase, these two functions are only used in test code. They are never called by any other module in the production code.

Suggestion:

It is recommended to remove these unused functions from the production code. If they are needed for testing only, they should be moved to the test code directly.

Resolution:

The team adopted our advice and fixed this issue by implementing the recommended fix, which can be found at [c4a5b7e4123256eeac424de274f061f1c0175a87](#).

FAR-6 Attackers Can Suppress Reward Distribution by Triggering Frequent Updates

Severity: Medium

Status: Fixed

Code Location:

contracts/farming/sources/farming.move#741-751

Descriptions:

In `farming.update_reward_config()`, the protocol calculates the accumulated reward increment (`acc_delta`) as follows:

```
let elapsed = (to_s - from_s) as u128;  
let reward = farming_math::checked_mul_u128(elapsed, cfg.reward_per_second as u128);  
let old_acc = cfg.acc_reward_per_share;  
let acc_delta = farming_math::mul_div_down(reward, ACC_PRECISION, total_boost_shares);
```

where `ACC_PRECISION` is `1e12`.

A precision-loss issue exists in this calculation. If `elapsed` is only **1 second** and `total_boost_shares` becomes sufficiently large, `reward * ACC_PRECISION` may be **smaller than** `total_boost_shares`. Since `mul_div_down()` performs floor division, `acc_delta` will be rounded down to **0**.

Although no reward is accumulated (`acc_delta == 0`), the timestamp is still advanced. As a result, the reward for that elapsed second is effectively discarded.

An attacker can repeatedly invoke `update_reward_config()` every second, ensuring that each update only covers a one-second interval. If the above condition holds, every invocation produces `acc_delta == 0`, while continuously advancing the reward timestamp.

Consequently, **all rewards generated during those intervals are permanently lost, preventing all stakers from receiving their expected rewards.**

Suggestion:

It is recommended to replace ACC_PRECISION (1e12) with 1e18.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

CHE-8 Manipulable On-Chain Price Weakens Withdrawal Slippage Protection

Severity: Medium

Status: Fixed

Code Location:

contracts/llv_ext_cetus/sources/cetus_hasui_entry.move#151-158

Descriptions:

In `withdraw_hasui_underlying()`, the protocol calls `cetus_vault_adapter::get_position_amounts()` to calculate `expected_sui` and `expected_hasui`, and then derives the slippage protection parameters based on these values.

```
let (expected_sui, expected_hasui) =
cetus_vault_adapter::get_position_amounts<CetusLP>(
    vault, clmm_pool, lp_amount,
);
let (min_sui, min_hasui) = cetus_vault_adapter::withdraw_min_amounts(
    expected_sui,
    expected_hasui,
    withdraw_slippage_bps,
);
```

However, both `expected_sui` and `expected_hasui` are calculated entirely on-chain using the current pool state, particularly `current_sqrt_price`. Since `current_sqrt_price` can be temporarily manipulated (e.g., through flash-loan-assisted price manipulation or short-lived swaps), the resulting `expected_sui` and `expected_hasui` may already reflect a manipulated price rather than the true market value.

```
public fun get_position_amounts<CoinTypeA, CoinTypeB, T>(
    vault: &Vault<T>,
```

```

pool: &Pool<CoinTypeA, CoinTypeB>,
lp_amount: u64,
): (u64, u64) {
    assert!(vector::length(&vault.positions) == 1, EPositionSizeError);
    let wrapped_position_nft = vector::borrow(&vault.positions, 0);
    let clmm_position_nft = borrow_clmm_position(wrapped_position_nft);
    let (tick_lower, tick_upper) = position::tick_range(clmm_position_nft);
    let current_tick_index = pool::current_tick_index(pool);
    let current_sqrt_price = pool::current_sqrt_price(pool);
    let position_id = object::id(clmm_position_nft);
    let position_info = pool::borrow_position_info(pool, position_id);
    let total_liquidity = position::info_liquidity(position_info);
    let total_lp_amount = total_token_amount(vault);
    let delta_liquidity = get_share_liquidity_by_amount(
        lp_amount,
        total_liquidity,
        total_lp_amount,
    );

    let (amount_a, amount_b) = utils::get_amount_by_liquidity(
        tick_lower,
        tick_upper,
        current_tick_index,
        current_sqrt_price,
        delta_liquidity,
        false,
    );
    (amount_a, amount_b)
}

```

As a result, the subsequent slippage protection is based on compromised inputs instead of an independent price reference, significantly reducing its effectiveness. In extreme cases, an attacker could manipulate `current_sqrt_price` to influence the expected withdrawal amounts, causing inaccurate calculations and potentially bypassing the intended slippage protection.

Suggestion:

It is recommended to avoid deriving slippage protection solely from values calculated using the pool's current on-chain price.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LRE-10 Rebalance Withdraw Surplus Can Inflate Cached NAV

Severity: Major

Status: Fixed

Code Location:

contracts/liquidity_layer/sources/llv_rebalance.move#603

Descriptions:

The rebalance withdraw flow may double-count surplus assets after a protocol withdraw.

`settle_rebalance_withdraw()` calculates `accounted` as the minimum of amount, `current_balance`, and `remaining_plan`. It only deducts `accounted` from the source protocol cache, then sends any `surplus` to `idle`. If the real protocol position was reduced by the full `recalled` amount, the `surplus` is counted twice. It is added to `idle`, but it also remains inside the stale source protocol cached balance. This can inflate cached NAV until a later sync corrects the protocol balance. Several wrappers use this path without a post-withdraw sync, including `Scallop`, `Suilend`, `Cetus`, and `Vault` rebalance withdraw flows. `AlphaLend` and `NAVI` already resync the remaining live protocol balance after rebalance withdraw, which avoids this issue.

Cached NAV can be inflated by up to `REBALANCE_SYNC_TOLERANCE` for each affected rebalance withdraw leg. Later share pricing, `withdraw` planning, and allocation decisions may use unbacked assets before the next sync.

Suggestion:

It is recommended to resync the remaining live protocol balance after each rebalance withdraw, or deduct the full `recalled` amount from the source cached balance when `surplus` is credited to `idle`.

Resolution:

The team fixed the issue by deducting the `source_debit` amount from the source cached balance when `surplus` is credited to `idle`, which can be found at commit `c4a5b7e4123256eeac424de274f061f1c0175a87`.

AEN-11 Reward Compound Receipts May Allow Reward Diversion

Severity: Minor

Status: Acknowledged

Code Location:

contracts/llv_ext_alphalend/sources/alphalend_entry.move#869,978;

contracts/llv_ext_navi/sources/navi_entry.move#745;

contracts/llv_ext_suilend/sources/suilend_entry.move#854;

contracts/llv_ext_scallop/sources/scallop_entry.move#143

Descriptions:

The reward compound flow records the claimed reward amount in

`RewardCompoundReceipt` . `begin_reward_compound()` stores `claimed_reward_amount` , and `complete_reward_compound()` returns this value after checking only the pool id, protocol id, and sender. The completion wrappers discard `_claimed_reward_amount` and call `inject_reward_to_idle()` with the `Coin<Asset>` supplied by the keeper. There is no minimum received amount check. There is also no exchange-rate check or binding between the reward coin claimed in the begin step and the base asset injected in the complete step. As a result, a keeper can claim protocol rewards into the transaction, route the reward coin elsewhere, and complete the hot-potato receipt with a smaller unrelated base-asset coin. The pool treats the compound flow as completed and only credits the smaller injected amount to idle. Rewards that should increase NAV can be diverted.

A keeper can divert claimed protocol rewards instead of compounding them into the pool. This causes LPs to receive less NAV growth than expected.

Suggestion:

It is recommended to make completion consume a swap or output receipt that proves the claimed reward coin was converted, and to enforce a configured minimum received amount

before injecting assets into idle.

Resolution:

The team acknowledged this issue and stated that " `RewardCompoundReceipt` only binds to the `pool` , `protocol` , `sender` , and `amount` ; the completion path does not bind to the actual reward coin received, swap output, or injected assets. The current model treats the compound keeper as a fully trusted operational role responsible for swap and minimum recovery risk control, which is by design".

So no modifications will be made to the current version.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't pose any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

